How-To Library

Informatica

Reset Configuration Options in Test Data Management

© Copyright Informatica LLC 2015, 2021. Informatica LLC. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC. All other company and product names may be trade names or trademarks of their respective owners and/or copyrighted materials of such owners.

Abstract

This document explains how a reset operation works and how each configuration option impacts the reset operation. It also explains, with an example, when and why you should choose one configuration over the other. It assumes that you have knowledge of TDM and data sets in TDM.

Supported Versions

- Test Data Management 10.2.0 HotFix 2
- Test Data Management 10.2.0 HotFix 1
- Test Data Management 10.2.0
- Test Data Management 10.1.0
- Test Data Management 9.7.1 HotFix 1
- Test Data Management 9.7.1
- Test Data Management 9.7.0

Table of Contents

Overview. . . . 2
Terminology. . . . 3
How a Reset Operation Works. . . . 3
Reset Configuration Options. . . . 4
Data Set Example 1. . . . 5
Using Default Settings. . . . 6
Using The Truncate Tables Option. . . . 7
Using The Upsert in Transactional Tables Option. . . . 8
Data Set Example 2. . . . 10
Using Default Settings. . . . 11
Using The Truncate Tables Option. . . . 11
Using the Upsert in Transactional Tables Option. . . . 12
Impact of Disabling Constraints and Indexes During Reset. . . . 14
Conclusion. . . . 15

Overview

You can create multiple versions of data sets to store and manage test data in the test data mart.

You can reset a specific data set version to a target to return the test data in the target to a specific state. This is useful if the data gets corrupted, or if the data in the test environment has changed and you need old data to run tests.

Reset is the process of moving a specific data set version from the test data mart to a target. The target can be the test system where you need to replace the test data. Perform a reset to refresh a section of data in the source system. Test Data Management (TDM) includes multiple options and configuration settings that allow you to control the way the reset works.

Terminology

Before you perform a reset operation, you must understand terms related to data sets used in TDM.

This document uses the following terms related to data sets in TDM:

Data set

A set of related rows from one or more database tables. A data set can correspond to one or more test cases. Data in a data set is a slice of schema data that you use for certain test cases. You can create a data set before, during, and after running test cases.

Data set version

When you create another data set with the same name, you create another version of the data set. Versions of a single data set might contain the same data or different data.

Master table

Classify tables in which you share all records with other users as master tables. All ancestor tables of a master table are master tables.

Transactional table

A child table of a master table or a child table of other transactional tables. Classify tables with records that you do not share with other users as transactional tables. When you delete records in transactional tables you do not impact the work of other users.

Top-level transactional table

All descendant tables of a transactional table are transactional tables. A top-level transactional table is a direct child of a master table.

Cascade delete

Deletes the record and any related child records. During a reset operation, cascade delete starts from top-level transactional tables. Cascade delete ensures that additional records that are not in the data set version but that have ancestors in the data set version are also deleted.

Insert

Add a record that does not exist.

Upsert

Update a record if it exists, and insert the record if it does not exist.

How a Reset Operation Works

Perform a reset operation to return test data in a test environment to a specific state. Use the required version of a data set to reset the data in the test environment.

You must classify the tables in the data set version before you perform a reset operation. The table classification affects the results of a reset operation. The reset operation does not delete records in master tables. It only updates the records in master tables. The operation considers unclassified tables as transactional tables. During the reset, data in transactional tables can get deleted. If the table contains records that you share with other users, this could affect the work of other users.

A reset operation with the default configuration settings performs the following high-level steps:

1. Identifies all records in the target that are present in the data set version.
2. Creates a backup of the identified records.

3. Starting from top-level transactional tables, deletes the identified records and their child records in the target transactional tables.
4. Updates records that exist in both the data set version and the target in the target master tables.
5. Inserts records that exist only in the data set version master tables into the target master tables.
6. Inserts all records in transactional tables from the data set version to the target.
7. Deletes the backup.

After a reset operation with the default configuration settings, the target contains all the data in the data set version and any additional data that was in the target and not in the data set version.

Reset Configuration Options

You can perform a reset operation that uses the default settings or configure the reset operation to control the results of the reset.

You can select the following options to configure a reset operation:

- Truncate tables. Deletes all tables in the target and copies the data from the data set version. Returns the target schema to the exact same state as the data set version. Use this option if you want to restore the entire schema. For example, you can use this option if you are the only user of the schema or if you want to restore the schema for all users.
- Disable constraints and indexes. Disables the constraints and indexes before the reset operation and enables the constraints and indexes after the reset. This does not affect the results of the reset operation, but affects the performance.
- Upsert in transactional tables. Inserts records in transactional tables in the target if the records do not exist, and updates records that exist. When you select this option, you override the default behavior that only inserts records that do not exist in transactional tables.

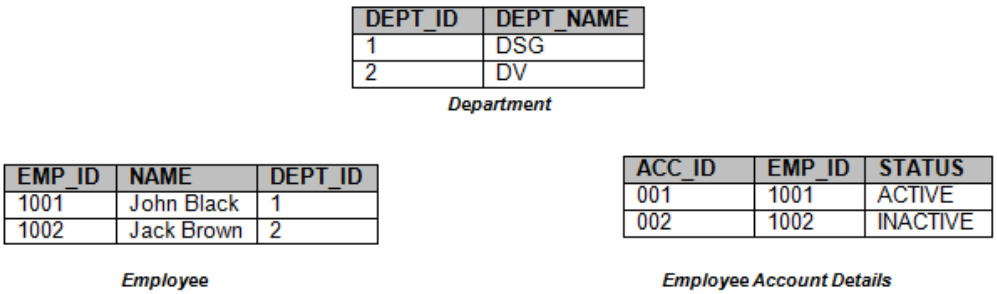
Configure a reset operation based on the data that exists in the target schema and the state to which you want to reset the target schema. A configuration option might not be the right option for a specific case. A reset operation might fail, you could lose data that you need, or you might not return the target to the required state if you do not configure the reset correctly.

Data Set Example 1

This example explains how the different reset configuration options can affect the final result and data in the target.

Consider the following data:

Figure 1. Data in Data Set Version



DEPT_ID	DEPT_NAME
1	DSG
2	DV

Department

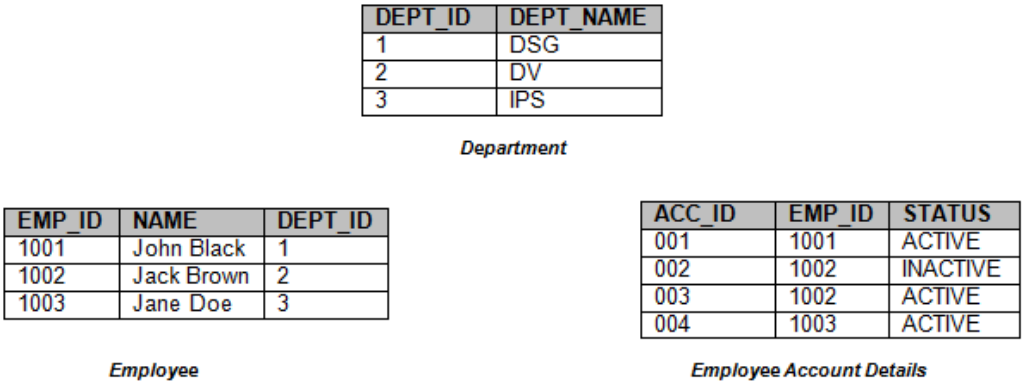
EMP_ID	NAME	DEPT_ID
1001	John Black	1
1002	Jack Brown	2

Employee

ACC_ID	EMP_ID	STATUS
001	1001	ACTIVE
002	1002	INACTIVE

Employee Account Details

Figure 2. Data in Target



DEPT_ID	DEPT_NAME
1	DSG
2	DV
3	IPS

Department

EMP_ID	NAME	DEPT_ID
1001	John Black	1
1002	Jack Brown	2
1003	Jane Doe	3

Employee

ACC_ID	EMP_ID	STATUS
001	1001	ACTIVE
002	1002	INACTIVE
003	1002	ACTIVE
004	1003	ACTIVE

Employee Account Details

Assumptions

This example considers the following assumptions about the data:

- Department is a master table.
- Employee is a transactional table.
- Employee Account Details is a transactional table.
- Employee Account Details is a child table of the Employee table.

The following sections in this example use this data to explain how the results of a reset operation might vary based on the configuration.

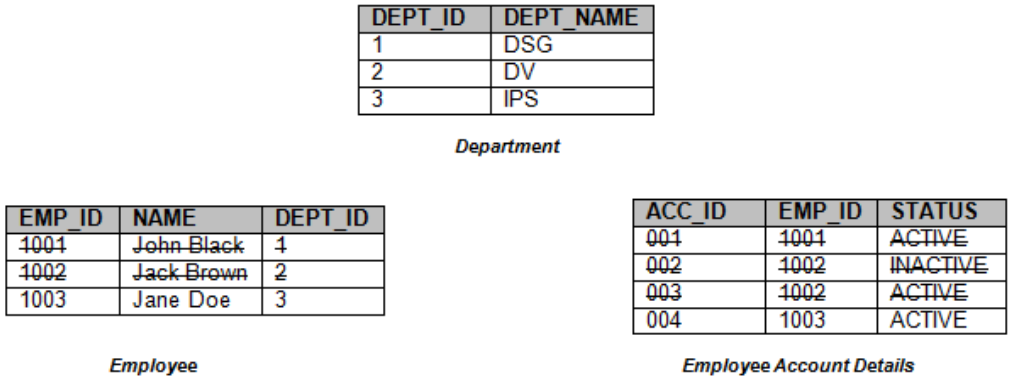
Using Default Settings

When you run the reset operation with the default settings, the operation performs the following steps:

1. Identifies and marks all records in the target that are present in the data set version.
2. Creates a backup of the marked records.
3. Beginning at the top-level transactional tables, deletes all the marked records and their child records.
4. Updates records that exist in both the data set version and the target in the target master tables.
5. Inserts records that exist only in the data set version master tables into the target master tables.
6. Inserts all records from the data set version in to the target transactional tables.
7. Deletes the backup.

The reset operation marks the following records in the target for deletion:

Figure 3. Target Records Marked for Deletion



DEPT_ID	DEPT_NAME
1	DSG
2	DV
3	IPS

Department

EMP_ID	NAME	DEPT_ID
~~1001~~	~~John Black~~	~~1~~
~~1002~~	~~Jack Brown~~	~~2~~
1003	Jane Doe	3

Employee

ACC_ID	EMP_ID	STATUS
~~001~~	~~1001~~	~~ACTIVE~~
~~002~~	~~1002~~	~~INACTIVE~~
~~003~~	~~1002~~	~~ACTIVE~~
004	1003	ACTIVE

Employee Account Details

- Does not mark any records in the Department table because no records in a master table are deleted.
- Marks the first two rows in the Employee table because they exist in the data set version.
- Marks the first two rows in the Employee Account Details table because they exist in the data set version.
- Marks the third row in the Employee Account Details table because the parent key record that the record points to in the Employee table, EMP_ID = 1002, is marked. Starting at the top-level transactional tables, the reset marks all related rows of rows marked for deletion.

When the reset operation completes, the target contains:

Figure 4. Target After Reset

DEPT_ID	DEPT_NAME
1	DSG
2	DV
3	IPS

Department

EMP_ID	NAME	DEPT_ID
1001	John Black	1
1002	Jack Brown	2
1003	Jane Doe	3

Employee

ACC_ID	EMP_ID	STATUS
001	1001	ACTIVE
002	1002	INACTIVE
004	1003	ACTIVE

Employee Account Details

The target contains all data that is in the data set version and any additional data with ancestors that are not in the data set version.

Using The Truncate Tables Option

Truncate tables when you want to reset the target to the exact same state as the data set version.

When you truncate tables in the reset operation, the reset performs the following steps:

1. Deletes all records in the target.
2. Inserts all records from the data set version in to the target.

When the reset operation completes, the target contains:

Figure 5. Target After Reset

DEPT_ID	DEPT_NAME
1	DSG
2	DV

Department

EMP_ID	NAME	DEPT_ID
1001	John Black	1
1002	Jack Brown	2

Employee

ACC_ID	EMP_ID	STATUS
001	1001	ACTIVE
002	1002	INACTIVE

Employee Account Details

The target and the data set version are identical when the reset operation completes.

Using The Upsert in Transactional Tables Option

You can choose to perform an upsert in transactional tables when you run the reset operation. By default, the reset operation only inserts records that do not exist in transactional tables. When you select this option, the reset operation inserts records that do not exist and updates records if they exist.

When you choose to upsert in transactional tables, the reset performs the following steps:

1. Identifies all records in the target transactional tables that are present in the data set version.
2. Creates a backup of the identified records.
3. Beginning at the top-level transactional tables, deletes all the identified records and their child records in the target.
4. Updates records that exist in both the data set version and the target in the target master tables.
5. Inserts records that exist only in the data set version master tables into the target master tables.
6. Inserts all deleted records from the data set version in to the target transactional tables.
7. Updates records in the target transactional tables that were not deleted during step 3.
8. Deletes the backup.

The following records are marked for deletion and then inserted from the data set version:

Figure 6. Target Records Marked for Deletion

DEPT_ID	DEPT_NAME
1	DSG
2	DV
3	IPS

Department

EMP_ID	NAME	DEPT_ID
~~1001~~	~~John Black~~	~~1~~
~~1002~~	~~Jack Brown~~	~~2~~
1003	Jane Doe	3

Employee

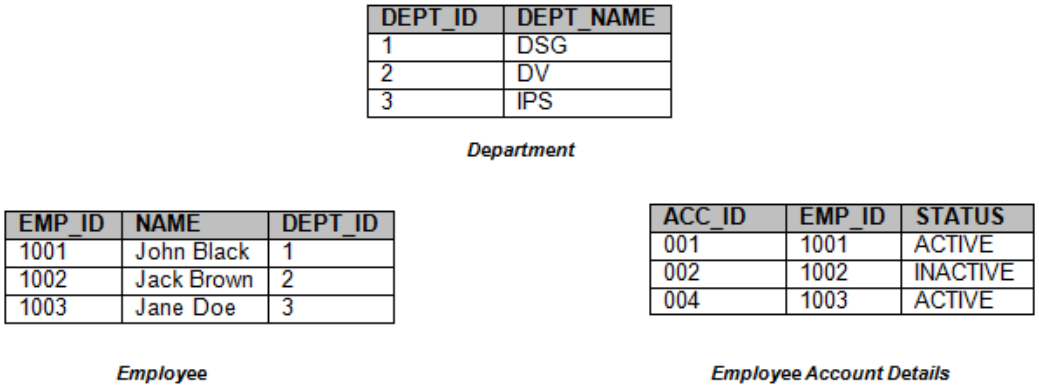
ACC_ID	EMP_ID	STATUS
~~001~~	~~1001~~	~~ACTIVE~~
~~002~~	~~1002~~	~~INACTIVE~~
~~003~~	~~1002~~	~~ACTIVE~~
004	1003	ACTIVE

Employee Account Details

- No records in the Department table are marked because no records in a master table are deleted.
- The first two rows in the Employee table are marked because they exist in the data set version.
- The first two rows in the Employee Account Details table are marked because they exist in the data set version.
- The third row in the Employee Account Details table is marked because the parent record that it points to in the Employee table, EMP_ID = 1002, is marked. Starting at the top-level transactional tables, all related rows of rows marked for deletion are marked.

When the reset operation completes, the target contains:

Figure 7. Target After Reset



DEPT_ID	DEPT_NAME
1	DSG
2	DV
3	IPS

Department

EMP_ID	NAME	DEPT_ID
1001	John Black	1
1002	Jack Brown	2
1003	Jane Doe	3

Employee

ACC_ID	EMP_ID	STATUS
001	1001	ACTIVE
002	1002	INACTIVE
004	1003	ACTIVE

Employee Account Details

The target contains all data that is in the data set version and any additional rows with ancestors that might have been in the target at the time of reset but not in the data set version.

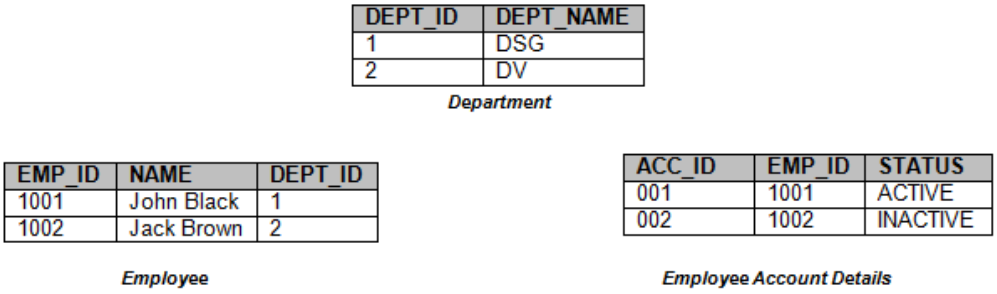
With this data, no records that are in both the data set version and the target remain after step 3 is performed. No changes were made to the parent records of records in the target after the data set version was created. In this situation, selecting the Upsert in Transactional Tables option is unnecessary as there are no records in transactional tables to update. The steps performed and the results are the same as when you perform a reset operation with the default settings.

Data Set Example 2

This example uses the data used in the first example, with some changes, to explain how the different configuration options affect the final result.

Consider the following data:

Figure 8. Data in Data Set Version



DEPT_ID	DEPT_NAME
1	DSG
2	DV

Department

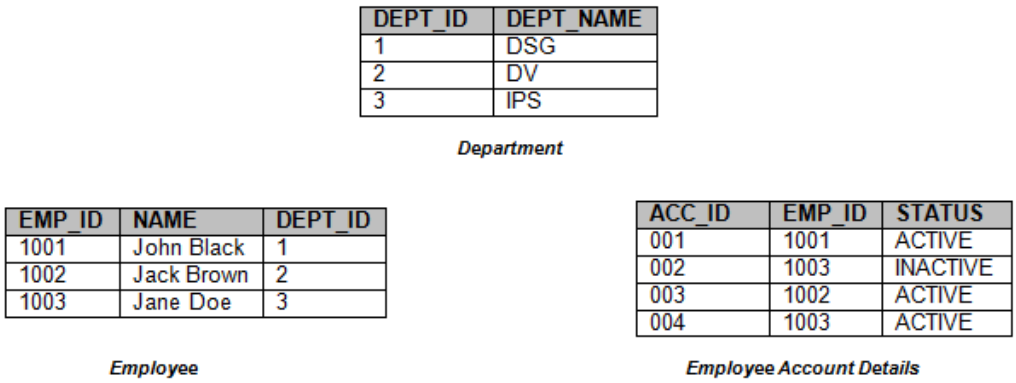
EMP_ID	NAME	DEPT_ID
1001	John Black	1
1002	Jack Brown	2

Employee

ACC_ID	EMP_ID	STATUS
001	1001	ACTIVE
002	1002	INACTIVE

Employee Account Details

Figure 9. Data in Target



DEPT_ID	DEPT_NAME
1	DSG
2	DV
3	IPS

Department

EMP_ID	NAME	DEPT_ID
1001	John Black	1
1002	Jack Brown	2
1003	Jane Doe	3

Employee

ACC_ID	EMP_ID	STATUS
001	1001	ACTIVE
002	1003	INACTIVE
003	1002	ACTIVE
004	1003	ACTIVE

Employee Account Details

The parent record of a record in the target was modified after the data set version was created. The parent record of a record in the Employee Account Details table (the second row), has changed to a different record from the Employee table. The previous parent record, EMP_ID = 1002, exists in the data set version. The current parent record, EMP_ID = 1003, does not exist in the data set version.

Assumptions

This example considers the following assumptions about the data:

- DEPARTMENT is a master table.
- EMPLOYEE is a transactional table.
- EMPLOYEE ACCOUNT DETAILS is a transactional table.
- EMPLOYEE ACCOUNT DETAILS is a child table of the EMPLOYEE table.

The following sections in this example explain how the results of the reset operation might vary based on the configuration.

Using Default Settings

When you run the reset operation with the default settings, the operation performs the following steps:

1. Identifies and marks all records in the target that are present in the data set version.
2. Creates a backup of the marked records.
3. Beginning at the top-level transactional tables, deletes all the marked records and their child records.
4. Updates records that exist in both the data set version and the target in the target master tables.
5. Inserts records that exist only in the data set version master tables into the target master tables.
6. Inserts all records from the data set version in to the target transactional tables.
7. Encounters a record in the target that exists in the data set version. This record was not marked in step 1 and therefore not deleted in step 3. The insert fails.
8. The reset operation fails.

The reset operation marks the following records for deletion:

Figure 10. Target Records Marked for Deletion

DEPT_ID	DEPT_NAME
1	DSG
2	DV
3	IPS

Department

EMP_ID	NAME	DEPT_ID
~~1001~~	~~John Black~~	~~1~~
~~1002~~	~~Jack Brown~~	~~2~~
1003	Jane Doe	3

Employee

ACC_ID	EMP_ID	STATUS
~~001~~	~~1001~~	~~ACTIVE~~
002	1003	INACTIVE
~~003~~	~~1002~~	~~ACTIVE~~
004	1003	ACTIVE

Employee Account Details

- No records in the Department table are marked because no records in a master table are deleted.
- The first two rows in the Employee table are marked because they exist in the data set version.
- The first row in the Employee Account Details table is marked because it exists in the data set version.
- The third row in the Employee Account Details table is marked because the parent key record that it points to in the Employee table, EMP_ID = 1002, is marked. Starting at the top-level transactional tables, all related rows of rows marked for deletion are marked.
- The second row in the Employee Account Details table is not marked because the current parent record in the Employee table, EMP_ID = 1003, is not in the data set version and not marked for deletion.

The reset operation fails and an error message displays.

With this data, you cannot perform a successful reset operation with the default settings.

Using The Truncate Tables Option

Truncate tables when you want to reset the target to the exact same state as the data set version.

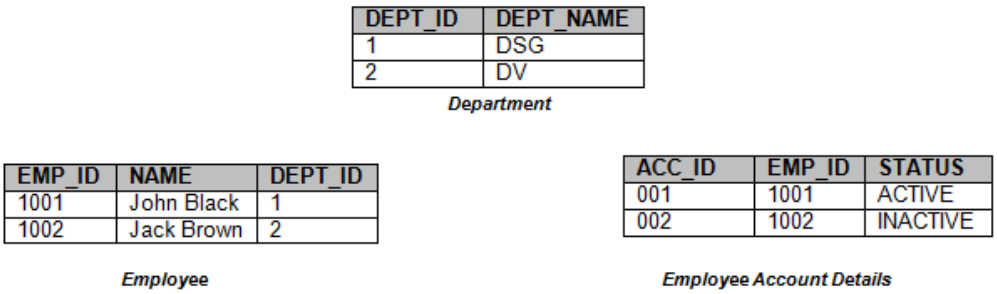
When you truncate tables in the reset operation, the reset performs the following steps:

1. Deletes all records in the target.

2. Inserts all records from the data set version in to the target.

When the reset operation completes, the target contains:

Figure 11. Target After Reset



DEPT_ID	DEPT_NAME
1	DSG
2	DV

Department

EMP_ID	NAME	DEPT_ID
1001	John Black	1
1002	Jack Brown	2

Employee

ACC_ID	EMP_ID	STATUS
001	1001	ACTIVE
002	1002	INACTIVE

Employee Account Details

The target and the data set version are identical when the reset operation completes.

The fact that the target contains a record that does not exist in the data set version does not impact the reset operation. This is because the reset operation first deletes all records in the target and then inserts all records from the data set version in to the target.

To restore the target schema to the same state as the data set version, you can use the truncate tables option even if the target contains changed data.

Using the Upsert in Transactional Tables Option

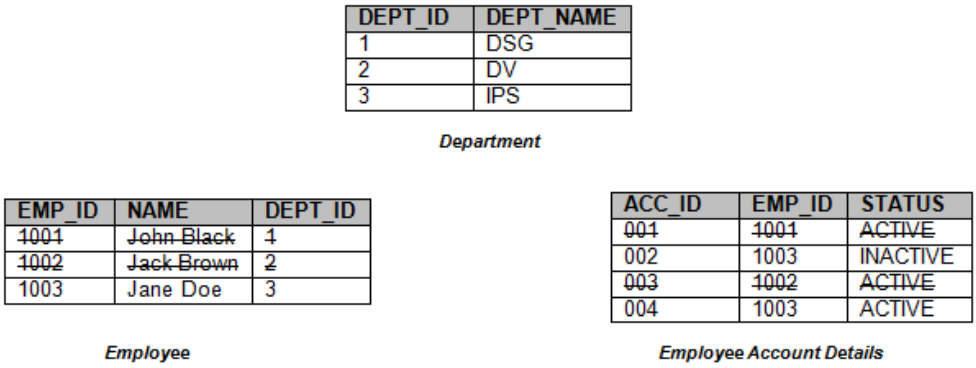
You can choose to perform an upsert in transactional tables when you run the reset operation. By default, the reset operation only inserts records that do not exist in transactional tables. When you select this option, the reset operation inserts records that do not exist and updates records if they exist.

The reset operation performs the following steps when you choose to upsert in transactional tables:

1. Identifies and marks all records in the target that are present in the data set version.
2. Creates a backup of the marked records.
3. Beginning at the top-level transactional tables, deletes all the marked records and their child records.
4. Updates records that exist in both the data set version and the target in the target master tables.
5. Inserts records that exist only in the data set version master tables into the target master tables.
6. Inserts all records from the data set version transactional tables in to the target.
7. Updates records in the target transactional tables that exist in the data set version but were not marked or deleted from the target.
8. Deletes the backup.

The reset operation marks the following records for deletion:

Figure 12. Target Records Marked for Deletion



DEPT_ID	DEPT_NAME
1	DSG
2	DV
3	IPS

Department

EMP_ID	NAME	DEPT_ID
~~1001~~	~~John Black~~	~~1~~
~~1002~~	~~Jack Brown~~	~~2~~
1003	Jane Doe	3

Employee

ACC_ID	EMP_ID	STATUS
~~001~~	~~1001~~	~~ACTIVE~~
002	1003	INACTIVE
~~003~~	~~1002~~	~~ACTIVE~~
004	1003	ACTIVE

Employee Account Details

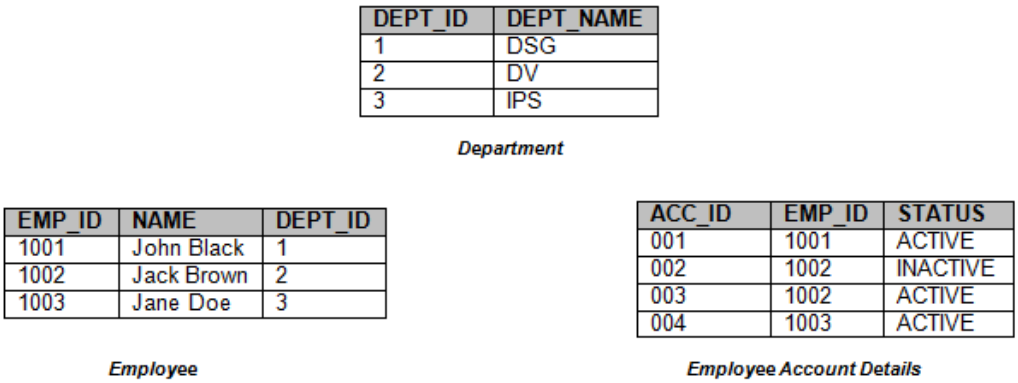
- No records in the Department table are marked because no records in a master table are deleted.
- The first two rows in the Employee table are marked because they exist in the data set version.
- The first row in the Employee Account Details table is marked because it exists in the data set version.
- The third row in the Employee Account Details table is marked because the parent key record that it points to in the Employee table, EMP_ID = 1002, is marked. Starting at the top-level transactional tables, all related rows of rows marked for deletion are marked.
- The second row in the Employee Account Details table is not marked because the current parent record in the Employee table, EMP_ID = 1003, is not in the data set version and not marked for deletion.

One record in the target table Employee Account Details that exists in the data set version is not deleted. With the default configuration that only inserts data, this reset operation would fail because of the existing record. The reset would fail when the operation tries to insert the row from the data set version.

When you configure a reset to upsert in transactional tables, the operation first verifies that the record exists, and updates the record if it does exist. It then inserts the records that do not exist (the records that were deleted).

When the reset operation completes, the target contains:

Figure 13. Target After Reset



DEPT_ID	DEPT_NAME
1	DSG
2	DV
3	IPS

Department

EMP_ID	NAME	DEPT_ID
1001	John Black	1
1002	Jack Brown	2
1003	Jane Doe	3

Employee

ACC_ID	EMP_ID	STATUS
001	1001	ACTIVE
002	1002	INACTIVE
003	1002	ACTIVE
004	1003	ACTIVE

Employee Account Details

The target contains all data that is in the data set version, and the additional data that is in the target at the time of reset but not in the data set version. The row in the target that was not deleted gets updated with data from the data set version.

With this data, a single record that is in both the data set version and the target remains after step 3. A change was made to the parent record of the record in the target after the data set version was created.

If the target contains records that you changed after you created the data set version, you can select the option to upsert in transactional tables. This ensures that the reset operation completes successfully. The default behavior of inserting records would cause the reset to fail.

Impact of Disabling Constraints and Indexes During Reset

You can configure a reset operation to disable the constraints and indexes during the operation.

When you disable the constraints and indexes, this configuration increases the performance of the reset operation. This configuration does not affect the results of a reset operation.

You can choose to disable constraints and indexes with any of the other configuration options. For example, you can disable constraints and indexes when you perform a reset with the default configuration. You can disable constraints and indexes when you override the settings to truncate tables, or override settings to perform an upsert on transactional tables.

When you disable the constraints, the reset performs delete and load in multiple tables in parallel. This increases the performance as the reset operation takes less time to complete.

When you do not disable constraints between tables during a reset operation, the reset operation must check the order in which deletes and inserts happen. The reset operation cannot perform load and delete for multiple tables in parallel, as it must consider constraints before deleting records. Such a reset operation might take longer to complete than when you disable constraints and indexes.

See Data Set Example 1, when you run a reset operation that uses the default settings. If you disable the constraints and indexes in this operation, during step 3, the reset deletes records in both transactional tables in parallel. If you do not disable the constraints and indexes, during step 3, the reset first deletes records in the child table and then deletes records in the parent table. The reset might take longer to complete. When you perform a reset on a data set that contains multiple child and parent tables, there might be a considerable difference in time taken when you disable constraints and when you do not.

You apply constraints between tables. When you disable constraints, you disable constraints for the entire table. During the time the constraints are disabled, tests that you run using the tables might fail as the constraints do not exist. Consider a case where multiple testers use the test data in the Customer table. You create a data set version that contains the first 100 records of the Customer table. After running a few tests, you want to reset the test data with the data set version. If you disable constraints for the table during the reset operation, tests run by other testers that include the Customer table fail. The reset operation causes a downtime for running tests that use the target connection.

While disabling constraints can increase performance, there might be a tradeoff with a downtime in testing if you share the target tables among testers.

If multiple testers use the tables, a slower reset operation might be better than a downtime in testing for multiple testers.

You can increase the performance of the reset by disabling constraints, if the following statements are true:

- You do not share the target tables with other users.
- The target tables are not used in tests during the reset operation.

Analyze the requirement and consider this tradeoff when you configure a reset operation.

You cannot choose to disable constraints and indexes instead of another configuration option to get the same results.

Consider the data in Example 2. The reset operation with default configuration fails because of a parent record or constraint in the target data that is not in the data set version. To ensure a reset operation completes successfully with this data, if you do not want to replace the target data completely, you must select the option to upsert in transactional tables. Disabling the constraints and indexes does not work as the operation fails when it tries to enable the constraints.

Conclusion

The examples show how different configuration settings in a reset operation can create different results.

Based on the data, some configurations might cause the reset operation to fail. You could configure the reset operation appropriately and perform a successful reset operation with the same data.

The configuration options that you use also depend on what you require from the reset. For example, to return a target to the exact same state as the data set version, you truncate tables. If you want to retain any additional data that is in the target and not in data set version, enabling the truncate tables option results in loss of required data.

It is important to understand what you require from a reset operation, understand the data, and understand how to configure the reset operation appropriately to get the required results.

Author

Sadhana Kamath

Acknowledgements

The author would like to acknowledge Anay Ghotikar for his technical assistance.